Functions

اَللّٰهُمَّ اُرْزُقْنِی عِلْمًا نَافِعًا وَاسِعًا عَمِيْقًا

اَللّٰهُمَّ اُرْزُقْنِى رِزْقًا وَاسِعًا حَلَالًا طَيِّبًا
مُبَارَكًا مِنْ عِنْدِكَ

What are functions

A function in programming is a reusable block of code that performs a specific task and can be executed whenever needed.

What are functions (An other definition)

A function is a named **set of instructions** that takes input (optional), processes it, and returns an output (optional).

Review the code
Mini UMS

Our code has grown to 360 lines, and everything is written inside the main() function.

It is difficult to write all the code in a single place.

```
24    // CRUD  Create, Read, Update, Delete
25    while (true)
26    {
27        // main header of ums
28        system("cls");
29        cout << "-----------------------------------------------------" << endl;
30        cout << "-------University Admission Management System -------" << endl;
31        cout << "-----------------------------------------------------" << endl;
32
33        cout << "User menu " << endl;
34        cout << "1 Admin " << endl;
35        cout << "2 Student" << endl;
36        cout << "3 to Exit " << endl;
37        cout << "Choose option : ";
38        string userOption;
39        cin >> userOption;
40
41        cout << "You choose " << userOption << endl;
42        if (userOption == "1")
43        {
44            // write here the admin code
45            for (int i = 0; i < 3; i++)
46            {
47                system("cls");
48                cout << "Admin Menu : Login attempt " << i + 1 << endl;
49                cout << "Enter username : ";
50                string username;
51                cin >> username;
52                cout << "Enter password : ";
53                string password;
54                cin >> password;
55                if (username == "admin" && password == "123")
```

How to manage growing code?

Our code can grow to hundreds, thousands, or even millions of lines, so it is not practical to maintain everything inside a single main() function. We need a way to break complex code into smaller, manageable, and reusable chunks.

This is where functions help us.

Types of Functions

- Built-in Functions
- User-Defined Functions

Pre-Defined Functions

- We can use library functions by invoking the functions directly; we don't need to write the functions ourselves.

- In order to use library functions, we usually need to include the header file in which these library functions are defined.

Pre-Defined Functions

- In C++, pre-defined functions are organized into separate libraries.
- For example, the header file iostream contains I/O functions; such as cout and cin functions.

```
1  #include <iostream>
```

Pre-Defined Functions

- Similarly, the header file cmath contains math functions; such as pow, sqrt, fabs and floor etc.

```
1  #include <iostream>
2  #include <cmath>
```

Pre-Defined Functions

FunctionType	Header File	Purpose	Parameter(s) Type	Result
pow(x, y)	<cmath>	Returns x^y; if x is negative, y must be a whole number pow(0.16, 0.5) = 0.4	double	double

Pre-Defined Functions

FunctionType	Header File	Purpose	Parameter(s) Type	Result
pow(x, y)	<cmath>	Returns x^y; if x is negative, y must be a whole number pow(0.16, 0.5) = 0.4	double	double
sqrt(x)	<cmath>	Returns the nonnegative square root of x; x must be nonnegative sqrt(4.0) = 2.0	double	double

Pre-Defined Functions

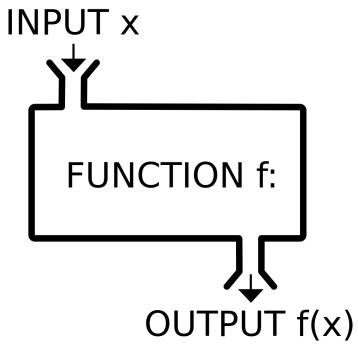
FunctionType	Header File	Purpose	Parameter(s) Type	Result
pow(x, y)	<cmath>	Returns x^y; if x is negative, y must be a whole number pow(0.16, 0.5) = 0.4	double	double
sqrt(x)	<cmath>	Returns the nonnegative square root of x; x must be nonnegative sqrt(4.0) = 2.0	double	double
fabs(x)	<cmath>	Returns the absolute value of its argument fabs(-5.67) = 5.67	double	double

Types of Functions

- **User-Defined Functions**
  Functions created by the programmer

Review

A function is a named **set of instructions** that takes input (optional), processes it, and returns an output (optional).



Understanding with example

Write a C++ program to manage the result of a student. The program should take the roll number and marks of three subjects as input. It should then calculate the total marks and percentage of the student. Based on the percentage, the program should assign a grade according to the following criteria: 80% and above for grade A, 70% to 79% for grade B, 60% to 69% for grade C, 50% to 59% for grade D, and below 50% for grade F. Finally, the program should display the roll number, total marks, percentage, and grade of the student in a clear format.

Understanding with example

```
1   #include <iostream>
2   using namespace std;
3
4   int main()
5   {
6       int roll, m1, m2, m3, total;
7       float percentage;
8       char grade;
9
10      // Input
11      cout << "Enter Roll No: ";
12      cin >> roll;
13
14      cout << "Enter marks of subject 1: ";
15      cin >> m1;
16      cout << "Enter marks of subject 2: ";
17      cin >> m2;
18      cout << "Enter marks of subject 3: ";
19      cin >> m3;
20
```

Understanding with example

```
21    // Calculation
22    total = m1 + m2 + m3;
23    percentage = total / 3.0;
24
25    // Grade calculation
26    if (percentage >= 80)
27        grade = 'A';
28    else if (percentage >= 70)
29        grade = 'B';
30    else if (percentage >= 60)
31        grade = 'C';
32    else if (percentage >= 50)
33        grade = 'D';
34    else
35        grade = 'F';
36
37    // Output
38    cout << "\nRoll No: " << roll;
39    cout << "\nTotal Marks: " << total;
40    cout << "\nPercentage: " << percentage << "%";
41    cout << "\nGrade: " << grade << endl;
```

```cpp
#include <iostream>
using namespace std;

main()
{
    int roll, m1, m2, m3, total;
    float percentage;
    char grade;

    // Input
    cout << "Enter Roll No: ";
    cin >> roll;

    cout << "Enter marks of subject 1: ";
    cin >> m1;
    cout << "Enter marks of subject 2: ";
    cin >> m2;
    cout << "Enter marks of subject 3: ";
    cin >> m3;

    // Calculation
    total = m1 + m2 + m3;
    percentage = total / 3.0;

    // Grade calculation
    if (percentage >= 80)
        grade = 'A';
    else if (percentage >= 70)
        grade = 'B';
    else if (percentage >= 60)
        grade = 'C';
    else if (percentage >= 50)
        grade = 'D';
    else
        grade = 'F';

    // Output
    cout << "\nRoll No: " << roll;
    cout << "\nTotal Marks: " << total;
    cout << "\nPercentage: " << percentage << "%";
    cout << "\nGrade: " << grade << endl;

}
```

Understanding with example

- In this code, we are performing multiple tasks inside a single main() function.
- This is fine for small programs, but we aim to develop complex and scalable software, where code should be easy to manage and extend.
- To achieve this, we divide the program into multiple functions.
- A fundamental rule of functions is: a function should perform only one task.
- Now, from the given code, we can identify how many different tasks are being performed and separate them into functions.

Identify the tasks

```
1   #include <iostream>
2   using namespace std;
3
4   main()
5   {
6       int roll, m1, m2, m3, total;
7       float percentage;
8       char grade;
9
10      // Input
11      cout << "Enter Roll No: ";
12      cin >> roll;
13
14      cout << "Enter marks of subject 1: ";
15      cin >> m1;
16      cout << "Enter marks of subject 2: ";
17      cin >> m2;
18      cout << "Enter marks of subject 3: ";
19      cin >> m3;
20
21      // Calculation
22      total = m1 + m2 + m3;
23      percentage = total / 3.0;
24
25      // Grade calculation
26      if (percentage >= 80)
27          grade = 'A';
28      else if (percentage >= 70)
29          grade = 'B';
30      else if (percentage >= 60)
31          grade = 'C';
32      else if (percentage >= 50)
33          grade = 'D';
34      else
35          grade = 'F';
36
37      // Output
38      cout << "\nRoll No: " << roll;
39      cout << "\nTotal Marks: " << total;
40      cout << "\nPercentage: " << percentage << "%";
41      cout << "\nGrade: " << grade << endl;
42
43  }
```

Understanding with example

These are the tasks that we are doing in the given code

1. Input Student Data
2. Calculate Total Marks
3. Calculate Percentage
4. Determine Grade
5. Display Result

Identify the tasks

```cpp
#include <iostream>
using namespace std;

main()
{
    int roll, m1, m2, m3, total;
    float percentage;
    char grade;

    // Input
    cout << "Enter Roll No: ";
    cin >> roll;

    cout << "Enter marks of subject 1: ";
    cin >> m1;
    cout << "Enter marks of subject 2: ";
    cin >> m2;
    cout << "Enter marks of subject 3: ";
    cin >> m3;

    // Calculation
    total = m1 + m2 + m3;
    percentage = total / 3.0;

    // Grade calculation
    if (percentage >= 80)
        grade = 'A';
    else if (percentage >= 70)
        grade = 'B';
    else if (percentage >= 60)
        grade = 'C';
    else if (percentage >= 50)
        grade = 'D';
    else
        grade = 'F';

    // Output
    cout << "\nRoll No: " << roll;
    cout << "\nTotal Marks: " << total;
    cout << "\nPercentage: " << percentage << "%";
    cout << "\nGrade: " << grade << endl;

}
```

Understanding with example

For each task we can write separate function

1. Function to Input Student Data
2. Function to Calculate Total Marks
3. Function to Calculate Percentage
4. Function to Determine Grade
5. Function Display Result

Writing your first function

Syntax

```
returnType functionName (parametersList) {

    //Body

    return value;
}
```

Writing your first function

void, int, float, string, char, bool, double, array, and other data types

```
returnType functionName (parametersList) {
    //Body

    return value;
}
```

Writing your first function

A function name can be any valid identifier, but it must follow the same rules as variable names in C++.

```
returnType functionName (parametersList) {

    //Body

    return value;
}
```

Writing your first function

A parameter list is a comma separated list of variables inside parentheses that defines the input values a function receives.

```
returnType functionName (parametersList) {

    //Body

    return value;
}
```

```
int add(int a, int b)
```

Writing your first function

A parameter list is a comma separated list of variables inside parentheses that defines the input values a function receives.

```
returnType functionName (parametersList) {

    //Body

    return value;
}
```

```
int add(int a, int b)
float average(int a, int b, int c)
```

Writing your first function

A parameter list is a comma separated list of variables inside parentheses that defines the input values a function receives.

```
returnType functionName (parametersList) {

    //Body

    return value;
}
```

```
int add(int a, int b)
float average(int a, int b, int c)
void printStudent(int roll, string name)
```

Writing your first function

The return statement is used in a function to send a value back to the caller and terminate the function execution.

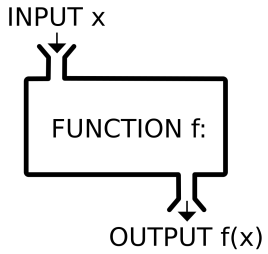
```
returnType functionName (parametersList) {
        //Body
    return value;
}
```

Writing your first function

The body of a function contains the set of statements that define what the function does and is written inside curly braces { }.

```
returnType functionName (parametersList) {
    //Body
    return value;
}
```

User-Defined Functions in C++



Return Type

Function Name

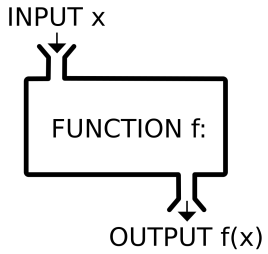
Input Parameters list

```
1  int addition(int num1, int num2)
2  {
3      int sum = num1 + num2;
4      return sum;
5  }
```

Body of the Function

returning value

User-Defined Functions in C++

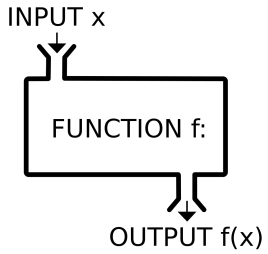



```
1  int addition(int num1, int num2)
2  {
3      int sum = num1 + num2;
4      return sum;
5  }
```

```
1  main(){
2      int number1, number2, result;
3      cout << "Enter First Number: ";
4      cin >> number1;
5      cout << "Enter Second Number: ";
6      cin >> number2;
7      result = addition(number1, number2);
8      cout << "Sum is: " << result;
9  }
```

Function Call

User-Defined Functions in C++

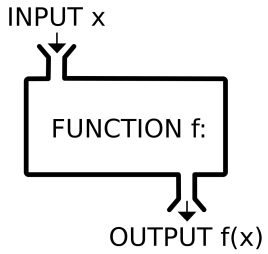



```
int addition(int num1, int num2)
{
    int sum = num1 + num2;
    return sum;
}
```

```
main(){
    int number1, number2, result;
    cout << "Enter First Number: ";
    cin >> number1;
    cout << "Enter Second Number: ";
    cin >> number2;
    result = addition(number1, number2);
    cout << "Sum is: " << result;
}
```

Parameter Passing

User-Defined Functions in C++




```
1  int addition(int num1, int num2)
2  {
3      int sum = num1 + num2;
4      return sum;
5  }
```

```
1  main(){
2      int number1, number2, result;
3      cout << "Enter First Number: ";
4      cin >> number1;
5      cout << "Enter Second Number: ";
6      cin >> number2;
7      result = addition(number1, number2);
8      cout << "Sum is: " << result;
9  }
```

Function returning the output

Identify the tasks

```cpp
#include <iostream>
using namespace std;

main()
{
    int roll, m1, m2, m3, total;
    float percentage;
    char grade;

    // Input
    cout << "Enter Roll No: ";
    cin >> roll;

    cout << "Enter marks of subject 1: ";
    cin >> m1;
    cout << "Enter marks of subject 2: ";
    cin >> m2;
    cout << "Enter marks of subject 3: ";
    cin >> m3;

    // Calculation
    total = m1 + m2 + m3;
    percentage = total / 3.0;

    // Grade calculation
    if (percentage >= 80)
        grade = 'A';
    else if (percentage >= 70)
        grade = 'B';
    else if (percentage >= 60)
        grade = 'C';
    else if (percentage >= 50)
        grade = 'D';
    else
        grade = 'F';

    // Output
    cout << "\nRoll No: " << roll;
    cout << "\nTotal Marks: " << total;
    cout << "\nPercentage: " << percentage << "%";
    cout << "\nGrade: " << grade << endl;

}
```

Understanding with example

For each task we can write separate function

1. Function to Calculate Total Marks
2. Function to Calculate Percentage
3. Function to Determine Grade
4. Function Display Result

Understanding with example

1. Function to Calculate Total Marks

```
// Calculate total marks
int calculateTotal(int m1, int m2, int m3) {
    return m1 + m2 + m3;
}
```

Understanding with example

1. Function to Calculate Percentage

```
// Calculate percentage
float calculatePercentage(int total) {
    return total / 3.0;
}
```

Understanding with example

1. Function to
   Determine Grade

```
// Calculate grade
char calculateGrade(float percentage)
{
    char grade;
    if (percentage >= 80)
        grade = 'A';
    else if (percentage >= 70)
        grade = 'B';
    else if (percentage >= 60)
        grade = 'C';
    else if (percentage >= 50)
        grade = 'D';
    else
        grade = 'F';

    return grade;
}
```

Understanding with example

1. Function Display Result

```
// Display result
void displayResult(int roll, int total, float percentage, char grade)
{
    cout << "\nRoll No: " << roll;
    cout << "\nTotal Marks: " << total;
    cout << "\nPercentage: " << percentage << "%";
    cout << "\nGrade: " << grade << endl;
}
```

Understanding with example

Functions are ready, now how to use them?

Understanding with example

Function is always call (use) by its name.

```
calculateTotal(m1, m2, m3);
```

Driver Code

```
main()
{
    int roll, m1, m2, m3, total;
    float percentage;
    char grade;
    // Input (kept in main)
    cout << "Enter Roll No: ";
    cin >> roll;

    cout << "Enter marks of subject 1: ";
    cin >> m1;
    cout << "Enter marks of subject 2: ";
    cin >> m2;
    cout << "Enter marks of subject 3: ";
    cin >> m3;

    // Function calls
    total = calculateTotal(m1, m2, m3);
    percentage = calculatePercentage(total);
    grade = calculateGrade(percentage);
    displayResult(roll, total, percentage, grade);
}
```

Driver Code

```
main()
{
    int roll, m1, m2, m3, total;
    float percentage;
    char grade;
    // Input (kept in main)
    cout << "Enter Roll No: ";
    cin >> roll;

    cout << "Enter marks of subject 1: ";
    cin >> m1;
    cout << "Enter marks of subject 2: ";
    cin >> m2;
    cout << "Enter marks of subject 3: ";
    cin >> m3;

    // Function calls
    total = calculateTotal(m1, m2, m3);
    percentage = calculatePercentage(total);
    grade = calculateGrade(percentage);
    displayResult(roll, total, percentage, grade);
}
```

Driver Code

```
main()
{
    int roll, m1, m2, m3, total;
    float percentage;
    char grade;
    // Input (kept in main)
    cout << "Enter Roll No: ";
    cin >> roll;

    cout << "Enter marks of subject 1: ";
    cin >> m1;
    cout << "Enter marks of subject 2: ";
    cin >> m2;
    cout << "Enter marks of subject 3: ";
    cin >> m3;

    // Function calls
    total = calculateTotal(m1, m2, m3);
    percentage = calculatePercentage(total);
    grade = calculateGrade(percentage);
    displayResult(roll, total, percentage, grade);
}
```

Where to write these functions?

You can define a function either **above** or **below** the main() function. However, if you define a function below main(), you must provide its **prototype (declaration)** before main() so the **compiler** knows about the function in advance.

Prototype of a function

```
1   #include <iostream>
2   using namespace std;
3
4    int add(int, int);  // Function prototype
5
6   int main() {
7       int result = add(5, 3);
8       return 0;
9   }
10
11  int add(int a, int b) {  // Function definition
12      return a + b;
13  }
14
```

Class Task

Give a task to the students to write a function that calculate the UET Aggregate. It takes ECAT marks, matric and FSC marks and return the final aggregate.

Syntax

```
// Calculate total marks
int calculateTotal(int m1, int m2, int m3) {
    return m1 + m2 + m3;
}
```

Class Task

A function that gets student marks array input and return maximum marks.

Class Task

A function that gets student marks array input and return maximum marks.

Class Task

```
6   int FindMaxNumber(int marks[], int size){
7       int max = marks[0];
8       for(int i=0; i< size; i++){
9           if(max < marks[i]){
10              max = marks[i];
11          }
12      }
13      return max;
14  }
```

Class Task

```
16  main(){
17
18      int marks[] = {10, 9, 2, 4, 1, 5, 3, 1};
19      int age[] = {15, 12, 14, 12};
20
21      int maxmarks = FindMaxNumber(marks, 8);
22      int maxAge = FindMaxNumber(age, 4);
23      cout<<" Max marks "<<maxmarks<<" ";
24      cout<<"Max age "<<maxAge;
25
26  }
```

Class Task

You have parallel arrays of student names and age.
You pass a single student name, and these arrays to the function and function return the age of that student if student not found it returns -1;

```
16  int FindStudentAge(string name, string names[], int ages[], int size){
17      for(int i =0; i<size; i++){
18          if(name == names[i]){
19              return ages[i];
20          }
21      }
22      return -1;
23  }
24
25  main(){
26
27      string names[] = {"ali", "ammar", "nasir"};
28      int ages[] = {10, 39, 34};
29
30      int age = FindStudentAge("ammar", names, ages, 3);
31      if(age == -1){
32       cout<<"Record not found ";
33      }else
34      cout<<"Age "<<age;
35
36  }
```

Single Responsibility Principle (SRP) of a Function

A function should have only one responsibility, meaning it should perform one specific task only.
It should not handle multiple unrelated jobs like input, processing, and output together. Instead, each function should focus on a single, clear purpose, such as calculating a value, displaying data, or validating input.

Question

This program defines a function studentResult() that calculates and displays a student's result. It takes marks of 3 subjects as input from the user, computes the total marks, calculates the percentage, determines the grade based on the percentage, and finally displays the total, percentage, and grade.

Identify the issue with this code?

```
void studentResult() {

    int marks[3];
    int total = 0;
    float percentage;

    cout << "Enter marks of 3 subjects:\n";
    for (int i = 0; i < 3; i++) {
        cin >> marks[i];
        total += marks[i];
    }
    percentage = total / 3.0;
    char grade;
    if (percentage >= 80)
        grade = 'A';
    else if (percentage >= 70)
        grade = 'B';
    else if (percentage >= 60)
        grade = 'C';
    else if (percentage >= 50)
        grade = 'D';
    else
        grade = 'F';

        cout << "\n===== RESULT =====\n";
    cout << "Total: " << total << endl;
    cout << "Percentage: " << percentage << endl;
    cout << "Grade: " << grade << endl;
}
```

Applying SRP

Function to take inputs in the array

```
1   #include <iostream>
2   using namespace std;
3
4   void inputMarks(int marks[], int size) {
5       cout << "Enter marks of 3 subjects:\n";
6       for (int i = 0; i < size; i++) {
7           cin >> marks[i];
8       }
9   }
10
```

Applying SRP

Function to calculate the total marks from the array

```
int calculateTotal(int marks[], int size) {
    int total = 0;
    for (int i = 0; i < size; i++) {
        total += marks[i];
    }
    return total;
}
```

Applying SRP

Calculating the percentage

```
19  float calculatePercentage(int total, int size) {
20      return total / (float)size;
21  }
22
```

Applying SRP

Calculating the Grade

```
23  char calculateGrade(float percentage) {
24      if (percentage >= 80)
25          return 'A';
26      else if (percentage >= 70)
27          return 'B';
28      else if (percentage >= 60)
29          return 'C';
30      else if (percentage >= 50)
31          return 'D';
32      else
33          return 'F';
34  }
```

Applying SRP

Displaying the values

```
36 void displayResult(int total, float percentage, char grade) {
37     cout << "\n===== RESULT =====\n";
38     cout << "Total: " << total << endl;
39     cout << "Percentage: " << percentage << endl;
40     cout << "Grade: " << grade << endl;
41 }
```

Applying SRP

Driver Code

```
43  main() {
44
45      int marks[3];
46      int total;
47      float percentage;
48      char grade;
49
50      inputMarks(marks, 3);
51
52      total = calculateTotal(marks, 3);
53      percentage = calculatePercentage(total, 3);
54      grade = calculateGrade(percentage);
55
56      displayResult(total, percentage, grade);
57
58  }
```

Learning Outcome

By the end of this lecture, students will be able to understand the concept of functions, identify different types of functions, and develop programs using user-defined functions to create modular, reusable, and well-structured code.

Self Assessment

1. What will be the output of the program?

```
#include <iostream>
using namespace std;

/* global variable declaration */
int g = 20;
main()
{
    /* local variable declaration */
    int g = 10;
    cout << "Value of g = " << g;
}
```

Self Assessment

2. What will be the sequence of the output of the program? How many global variables, local variables of main, parameters of sum function and local variables of sum function are there?

```
#include <iostream>
using namespace std;

int a = 20;
int sum(int a, int b);
main ()
{
  int a = 10;
  int b = 20;
  int c = 0;
  cout << "value of a in main() = " << a << endl;
  c = sum( a, b);
  cout << "value of c in main() = " << c << endl;
}
/* function to add two integers */
int sum(int a, int b)
{
   cout << "value of a in sum() = " << a << endl;
   cout << "value of b in sum() = " << b << endl;
   return a + b;
}
```

Self Assessment

Explain this code

```
#include <iostream>
using namespace std;

float calculateDiscount(float price, float discountPercent) {
    return (price * discountPercent) / 100;
}

float finalPrice(float price, float discount) {
    return price - discount;
}

void displayResult(float price, float discount, float final) {
    cout << "\n--- Result ---" << endl;
    cout << "Original Price = " << price << endl;
    cout << "Discount Amount = " << discount << endl;
    cout << "Final Price = " << final << endl;
}

main() {
    float price, discountPercent;

    cout << "Enter original price: ";
    cin >> price;

    cout << "Enter discount percentage: ";
    cin >> discountPercent;

    float discount = calculateDiscount(price, discountPercent);
    float final = finalPrice(price, discount);
    displayResult(price, discount, final);

}
```

Self Assessment

Convert your Mini UMS App into the functions